next-workflow 2.1.1: Review Update

Where every v2 finding stands after the fixes, what the changes cost in tokens, and what's still open.

Updated 15 Oct 2026 for kit v2.1.1 (git tag `v2.1.1`). It replaces the v2 review of the same day; the original v1 review is unchanged in `next-workflow-review.pdf`. ✓ **VERIFIED** means reproduced by running real tools. Token figures are ≈ **characters ÷ 4**, a standard estimate that's accurate to about ±10% for English prose and code.

1. Verdict

4.1

out of 5 (v2.0: 3.7 · v1: 3.4)

Solid and safe enough to use day to day. Its real-world pass rate is still unmeasured.

13 of the 15 v2 findings are fixed and covered by tests (284 assertions, shellcheck clean, CI defined). One more (N2) is half done, and one known limit remains (R2). The riskiest fixes were proven with real tools: Playwright 1.64.1 in a browser, and a real Next 15.5.4 build and server for the eval checks. Two rounds of automated security review found 8 more issues; all were fixed except one class that a text-based guard can't catch, which is documented (R2).

On tokens: a single task now costs a bit more, because an independent agent checks it. A batch of 3 or more tasks costs noticeably less, because each task starts in a fresh session instead of one ever-growing conversation. The fixed overhead per session is almost unchanged (+30 tokens). Details in section 3.

Scorecard: v1 → v2.0 → v2.1.1

Dimension	v1	v2.0	v2.1.1		Why it moved
Architecture & flow	4.0	4.5	4.5		Unchanged: plan → approve → build → verify → checkpoint.
Token efficiency	4.5	4.0	4.0		The verifier costs more per task; fresh sessions save on batches; the skip rule avoids the verifier on trivial tasks.
Code-quality guardrails	4.0	3.5	4.0		Shell-change check now blocks only new violations (N4).
Security	2.5	3.5	4.0		Checkpoints never commit secrets; over-broad permission rules removed; guard hardened. A text guard has limits (R2).
Reliability & recovery	2.5	4.0	4.5		Dependency-aware undo, batch branches.
Verification depth	3.0	3.5	4.0		Browser checks proven to run; verdicts recorded.
Framework currency	2.5	3.5	4.5		Next 16 rules checked against the official docs; Claude is pointed at the docs bundled with the installed Next.

Maintainability	3.5	3.0	4.0		Git, changelog, CI, shellcheck-clean.
Observability	1.5	3.0	4.0		Verifier first-pass rate, edit-hook speed, runner vs in-session task counts.
Proven outcomes	n/a	1.5	2.5		The eval checks are proven against a reference (5/5, plus 5 deliberate breakages caught); agent runs still pending (R1).

2. Status of every v2 finding

ID	Finding	Status	What was done · evidence
N1	A checkpoint commit can capture secrets	FIXED ✓	Only the task's own files are committed; pre-existing edits are left out; changed secret files block the checkpoint. Batches commit on `nw/batch-*`. Tested: `.env`, pre-existing `notes.txt` and the cache are never committed.
N2	The full loop has never run	PARTLY	The harness is proven end to end: a known-good solution scores 5/5 with a real build and `next start`, and 5 deliberate breakages are each caught. **Agent runs not done yet:** they cost API money and need your go-ahead (R1).
N3	Browser verification probably can't run	FIXED ✓	Confirmed the bug with real Playwright 1.64.1 ("No tests found"). The kit's own config, run through the `verify-browser.sh` wrapper, passes in a real browser.
N4	Shell check blocks over old problems	FIXED ✓	Each file is compared with its committed version, and only new violations block. Tested both ways.
N5	Secrets guard misfires	FIXED ✓	`.env.example` is readable; safe `.gitignore` and seed commands are allowed. Hardened further after two security-review passes (section 4).
N6	Undo ignores dependent tasks	FIXED	Undo refuses and names the dependents; `--cascade` undoes them newest first. Tested.
N7	Verifier verdicts aren't recorded	FIXED	`tasks.sh verdict` records each round; `stats` shows the first-pass rate.
N8	Next 16 rules unchecked	FIXED ✓	Rewritten from 9 nextjs.org pages, with sources. 4 errors corrected; the Next 16 removal of "First Load JS" handled.
N9	No version control or CI	FIXED	Git repo, tags `v2.1.0` and `v2.1.1`, CHANGELOG, CI (shellcheck, self-test, reference eval).
N10	Eval too thin	FIXED ✓	HTTP behaviour checks, hidden tests (including an authorization test with a positive control), an ambiguous task, multi-run averages, a Next 16 variant. All pinned versions confirmed to exist on npm.
N11	Verifier skip rule subjective	FIXED	`tasks.sh can-skip-verify`: all `[review]` and 20 changed lines or fewer (kit bookkeeping excluded).
N12	Memory file cut off silently	FIXED	Cut at a line boundary, with a "prune it" warning.
N13	Per-edit delay growing	FIXED	Measured on every edit; `stats` shows p50/p95.

N14	Global install second-class	FIXED	Helper path in the session report, matching permission rules, full task cycle tested.
N15	Limits that rely on the model	FIXED	In-session vs runner starts measured; a nudge after 3 in 3 hours.

3. Does it use more tokens now?

> **Short answer:** the fixed overhead is nearly the same. **One task on its own costs roughly 10–15% more**, because the independent verifier runs. **Batches of 3 or more tasks cost roughly 20–40% less**, because each task starts fresh instead of carrying every earlier task in context. Projects that already have type errors save the most.

3a. Fixed cost: measured

This is the text loaded into Claude's context. The v1 numbers are estimated from the line counts recorded during the v1 review (v1 has no git history).

What	When it's loaded	v1	v2.1.1	Change
Session report (stack + memory)	every session	≈ 340	365	+25
`/next:task` instructions	every task run	≈ 2,330	2,485	+155
`/next:feature` phases	large tasks only	≈ 1,000	1,085	+85
`/next:write-task` (planner)	once per plan	≈ 955	1,109	+154
`next-reviewer` prompt	reviews	≈ 1,100	1,062	−40 (duplicate list items merged)
`next-verifier` prompt	non-trivial tasks	—	811	new
`next16.md` reference	Next 16 projects, only for caching/proxy/config work	—	2,550	new, conditional
Per-edit check output	after edits with findings	up to 4 messages	1 merged message	slightly less
Bookkeeping calls (`can-skip-verify` , `verdict`)	per task	—	≈ 100–200	new, small

Net fixed overhead per task: about +200 tokens. That's negligible next to the work a task does (typically 20,000–40,000 tokens of file reads and tool output).

3b. Variable cost: what really moves the total

Change	Effect on tokens	Size
Independent verifier (H5)	MORE It's a separate agent that re-reads the task, the diff and the touched files, and runs tests.	≈ 10k–25k per non-trivial task; skipped for small all-`[review]` tasks; a FAIL round repeats it
Fresh session per task (H3, via `run-tasks.sh`)	LESS Each task no longer resends the context left over from earlier tasks on every turn. It re-pays only the fixed start-up text.	Grows with batch size: about −20% at 3 tasks, −40% at 5, more after that
Type-error baseline (M2)	LESS In projects with existing errors, v1 re-sent up to 40 lines of old errors at the end of every turn, and the agent often tried to fix them.	≈ 600–1,000 tokens per turn, plus avoided detours
Fewer false blocks (M1, N4)	LESS Each wrong block cost a round-trip and a workaround.	A few thousand tokens per avoided block
Two-round fix limit, then block	LESS Stops an agent looping on a criterion it can't meet.	Puts a ceiling on bad cases
Planner uses Opus (model split)	Same tokens, higher price per token	Only the planning step; building runs on Sonnet

3c. Batch model (illustrative, not measured)

These are clearly labelled assumptions, chosen to show the shape of the trade-off, not exact numbers:

- ~18k tokens of fixed context per session (Claude Code + session report + task instructions);
- 20 model turns per task;
- context grows ~30k over a task;
- an in-session batch carries ~15k per finished task into the next one;
- a verifier run ≈ 84k token-reads (6 turns × ~14k).

"Token-reads" counts every token sent to the model across all turns. Most of these are cache hits, which are billed at about 10% of the normal input price.

Tasks	v1 (one session)	v2.1.1 (runner + verifier)	Change
1	≈ 660k	≈ 744k	+13%
2	≈ 1.62M	≈ 1.49M	−8%
3	≈ 2.88M	≈ 2.23M	−22%
5	≈ 6.30M	≈ 3.72M	−41%
10	≈ 20.1M	≈ 7.44M	−63%

Two caveats:

- In practice v1 couldn't run 10 tasks in one session: compaction would have dropped detail, which is a quality problem as well as a cost problem.
- Fresh sessions re-write the prompt cache for the fixed context once per task, which costs a little more per token than cache reads. That's small next to the savings from 3 tasks up.

The real numbers come from the eval (R1): `tasks.sh stats` and `evals/results/*.json` record the cost per task.

3d. Ways to cut tokens further if needed

- **Verifier only on large tasks:** a setting (for example `NEXT_WF_VERIFY_SCOPE=large`) would restore v1's per-task cost on small tasks. Recommended only after the eval shows small tasks pass first time ≥90% of the time.
- **Haiku for the verifier on `[review]`-only tasks:** cheaper reading. Decide from the first-pass data in `stats`, not by guessing.
- **Keep `project-context.md` short:** it's loaded every session, and the truncation warning (N12) tells you when to prune.

4. Security review follow-ups (v2.1.1)

Automated security reviews of the v2.1.0 and v2.1.1 commits flagged 8 more issues. All were reproduced and fixed, except the last class below:

Issue	Fix
`git diff:*`, `git log:*` pre-approved (`--no-index` / `--output` reach files outside the deny rules)	Removed. Task diffs go through `tasks.sh diff`; `git status` is now exact.
`npx playwright test:*` pre-approved (`-c` / `--reporter` load any file as code)	Replaced by the `verify-browser.sh` wrapper, which accepts only a task id.
Exception bypasses: a new line, `$(…)` or backticks riding on an allowed command	The exceptions apply only to plain commands, matched as a whole with bash's own regex.
Glob, quote and backslash tricks (`.en*`, `.e[n]v`, `.en''v`, `~/.ss?/id_*`)	The command is normalized first; globs are matched against secret file names.
Filenames built at run time (`X=.e; cat ${X}nv`, `'.'+'env'` in a script)	ACCEPTED LIMIT No check that only reads the command text can catch these. Documented in the guard. See R2.

5. Still open

R1 The agent eval hasn't been run (rest of N2)

Why It's the only way to know the pass rate and the real cost per task, and to confirm the token model in section 3.

Fix `bash evals/run.sh` (3 runs; worst case ≈ $75 with the default $5 cap per task) or `--runs 1 --budget 2` (≤ $10). Commit the result file as the baseline.

R2 The secrets guard is a speed bump, not a boundary

Why Names built while a command runs, scripts, and recursive `grep` can still read `.env`. No check that only reads the command text can prevent this.

Fix Keep production secrets out of the project folder (use a secrets manager or a `.env.local` with development values only), and run unattended batches in a container or VM.

R3 **The Next 16 eval variant has no reference solution**

Why The Next 15 checks are proven against a reference; the Next 16 + `cacheComponents` run isn't yet, so a failure there could be a harness problem.

Fix Add `evals/reference-next16/` (Suspense around request data, `'use cache'` where needed) and allow `--reference --next 16`.

R4 **The verifier's cost isn't adjustable**

Why About 10–25k tokens per non-trivial task, whatever the project's risk profile.

Fix Add `NEXT_WF_VERIFY_SCOPE=all|large` and a model choice for `[review]`-only tasks, then tune both from the first-pass data.

R5 **Headless runs refuse commands that aren't pre-approved**

Why In `acceptEdits` mode, a command outside the allow list is refused rather than prompted. That's safe, but the agent may quietly use a weaker approach.

Fix The eval will show this. If needed, add narrowly scoped allow rules rather than broad prefixes.

R6 **Size**

Why There are now 15 scripts and 284 assertions. Each fix added surface, and the per-edit check takes ~0.65 s before formatting (measured).

Fix Watch `stats` (p95 edit-hook time). If it goes past ~2 s on real repos, consolidate the guards into one Node script.

Next step: run the eval once (R1). It turns the token model in section 3 into measured numbers, gives you a baseline to compare every future change against, and is the last thing standing between "well built" and "proven".